

1. Consider a 5×7 image stored in row-major order in memory, starting at address `img_ptr`. A Triton kernel processes this image using block sizes `bs0=2` (rows) and `bs1=3` (columns).

For the program with `pid_0 = 2`, `pid_1 = 2`:

- Compute the 1D offset vectors `offs_0` and `offs_1`. [2 pts]
- Compute the 2D offset matrix `offs = w * offs_0[:, None] + offs_1[None, :]`. Show the resulting 2×3 matrix. [1 pt]
- Compute the 2D mask matrix: `mask = (offs_0[:, None] < h) & (offs_1[None, :] < w)` where `h=5`, `w=7`. Show the resulting 2×3 boolean matrix and briefly explain which entries are masked out and why. [1 pt]
- Which image pixel(s) will this program actually load and process? Give the (row, col) coordinates. [1 pt]

2. You are given an $H \times W$ matrix stored in row-major order. Complete the Triton kernel below that computes its transpose, producing a $W \times H$ output matrix. Fill in the two blanks.

```
def transpose_k(in_ptr, out_ptr, H, W,
               bs0: tl.constexpr, bs1: tl.constexpr):
    pid_0 = tl.program_id(0)
    pid_1 = tl.program_id(1)

    offs_0 = pid_0 * bs0 + tl.arange(0, bs0)
    offs_1 = pid_1 * bs1 + tl.arange(0, bs1)

    load_offs = _____(a)_____
    store_offs = _____(b)_____

    mask = (offs_0[:, None] < H) & (offs_1[None, :] < W)

    data = tl.load(in_ptr + load_offs, mask=mask)
    tl.store(out_ptr + store_offs, data, mask=mask)
```

- Fill in blank (a): the expression for `load_offs`. [1 pt]
- Fill in blank (b): the expression for `store_offs`. [2 pts]
- Complete the launcher function by filling in the three blanks. [2 pts]

```
def transpose(x, bs=(32, 32)):
    H, W = x.shape
    out = torch.empty((W, H), dtype=x.dtype, device=x.device)
    grid = _____(c)_____
    transpose_k(grid](x, out, H, W,
                   bs0=____(d)____, bs1=____(e)____)
    return out
```

3. Consider the following CUDA kernel that applies an activation function:

```
__global__ void activation(float *x, float *out, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        if (x[i] > 0) {
            out[i] = x[i];
        } else {
            out[i] = 0.01f * x[i];
        }
    }
}
```

The kernel is launched with 256 threads per block on an array of 10240 floats containing a mix of positive and negative values.

- (a) Do you see a potential performance problem with this kernel? Explain. [2 pts]
 - (b) Would sorting the input array by sign (all negatives first, then all positives) improve the performance of this kernel? Explain briefly. [1 pt]
4. Consider the following CUDA kernel. Each particle has 4 attributes: x , y , z , w .

```
#define BS 32
__global__ void weighted_dist(float *positions, float *output, int N) {
    __shared__ float pos[BS][4];
    int tx = threadIdx.x;
    int i = blockIdx.x * BS + tx;
    // Load particle into shared memory
    if (i < N) {
        pos[tx][0] = positions[i * 4];
        pos[tx][1] = positions[i * 4 + 1];
        pos[tx][2] = positions[i * 4 + 2];
        pos[tx][3] = positions[i * 4 + 3];
    }
    __syncthreads();
    // Compute weighted distance from origin
    if (i < N) {
        float x = pos[tx][0];
        float y = pos[tx][1];
        float z = pos[tx][2];
        float w = pos[tx][3];
        output[i] = w * sqrtf(x*x + y*y + z*z);
    }
}
```


The kernel is launched with 32 threads per block. Shared memory has 32 banks, each 4 bytes wide.

- (a) Do you see a performance problem with the shared memory accesses in this kernel? [3 pts]
Explain clearly.
- (b) How would you fix this problem? Explain clearly. [3 pts]
5. Consider the following CUDA kernel that scales the position of each particle by a factor s .

```
struct Particle {  
    float x, y, z, w;  
};  
  
__global__ void scale_positions(Particle *particles, float s, int N) {  
    int tid = blockIdx.x * blockDim.x + threadIdx.x;  
    if (tid < N) {  
        particles[tid].x *= s;  
        particles[tid].y *= s;  
        particles[tid].z *= s;  
    }  
}
```

The kernel is launched with 32 threads per block.

- (a) Do you see a performance problem with the global memory accesses in this kernel? [3 pts]
Explain clearly.
- (b) How would you fix this problem? Show the modified code and explain why it works. [3 pts]
6. Consider a tiled matrix multiplication kernel that computes $C = A \times B$, where A is $M \times K$, B is $K \times N$, and the tile size is $T \times T$. Each output tile of C iterates over K/T chunks, and in each iteration loads a $T \times T$ tile from A and a $T \times T$ tile from B from global memory into shared memory.

The target GPU is an NVIDIA V100 with:

- Peak compute: 15.7 TFLOP/s (single precision)
 - Peak memory bandwidth: 898 GB/s
 - Shared memory per SM: 96 KB
- (a) Derive the arithmetic intensity (FLOP/byte) of this tiled matmul kernel in terms of T . Clearly show how you count the total FLOPs and total bytes transferred from global memory. [3 pts]
- (b) For tile size $T = 32$, compute the expected performance (GFLOP/s) using the roofline model. Is the kernel compute-bound or memory-bound on the V100? [2 pts]

- (c) What minimum tile size T is needed to make the kernel compute-bound? Do you foresee any practical problem with using that tile size? [3 pts]

7. A GPU application processes $N = 8$ data chunks. Each chunk requires the following stages:

Stage	Time
Read from disk to CPU memory	3 ms
Transfer CPU $\rightarrow$ GPU (H2D)	2 ms
GPU compute	4 ms
Transfer GPU $\rightarrow$ CPU (D2H)	2 ms

Assume all stages for different chunks can overlap with each other (using multiple CUDA streams, async transfers, and a separate I/O thread), but the **same stage cannot run on two chunks simultaneously** (e.g., only one disk read at a time, one H2D transfer at a time, etc.).

- (a) What is the total time to process all 8 chunks **without** any overlap (sequential execution)? [2 pts]
- (b) What is the total time with **full pipelining**? Show the schedule for the first 4 chunks to justify your answer. [3 pts]
- (c) What is the bottleneck stage in the pipeline? If you could speed up one stage, which one would you choose and why? [2 pts]